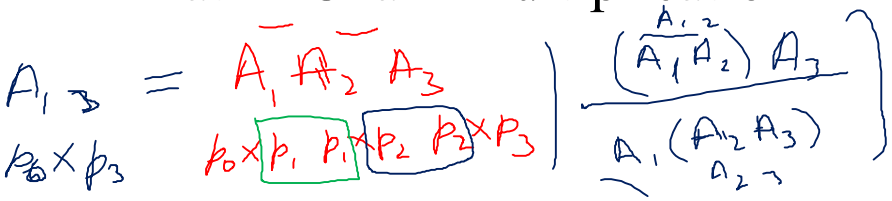
Matrix Chain Multiplication



1

Matrix Chain Multiplication

Problem

Given a sequence of matrices

$A_1, A_2, A_3, \ldots A_n$

with dimensions

$p_0 \times p_1, \quad p_1 \times p_2, \quad p_2 \times p_3, \ldots p_{n-1} \times p_n$

the problem is to find the order of multiplication that ***minimizes the number of multiplication and summation operations***.

- The problem does not consider the actual values stored in the sequence of matrices. It concerns the total number of operations , ***irrespective of the contents of matrices***.

2

Matrix Operations

Multiplications

A pair of ***rectangular matrices*** *A* and *B* can be multiplied if number of columns of A is equal to the number of rows of *B*. For example, if *A* has dimensions *p x q,* and B has dimension *q x r,* then since the number rows in *A* is *q* and number of columns in B is also *q*, the product AB can be computed. However, if B has dimensions *s x q,* then A and B cannot be multiplied.

- Let ***A*** and *B* be two rectangular matrices and let C be the product of *A* of and *B*, i.e, *C=A x B*
The ***rule for multiplication*** is as follows:

> *The element in* ***ith row and j th column of C*** *is obtained by multiplying the elements in the* ***i^{the} row of A*** *with the corresponding elements in the* ***jth column of B,*** *and then* ***summing*** *over all pairs of multiplications.*

- Let *A* be *p x q* matrix and *B* be *q x r* matrix, and *C= A x B.* The matrix *C* has the dimensions *p x r.*
The rule for matrix multiplication is expressed as the ***mathematical formula***:

$$c_{ij} = \sum_{k=1}^{q} a_{ik}.b_{kj} \qquad 1 \le i \le p, \quad 1 \le j \le r$$

3

Matrix Multiplication

Properties

- If *A* has dimensions *p x q* and *B* has dimensions *q x r*, then the product, *A x B* ,has dimensions *p x r*

- If *A* has dimensions *p x q* and *B* has dimensions *q x r*, then the product, *A x B* , is obtained by performing *p x q x r* multiplications and summations

- Matrix multiplication is ***associative***. This property implies that *(AB)C= A(BC)*.

- The number of multiplication and summation operations for computing the product *(AB)C* is ***different*** from number of operations for computing *A(BC)*. For example, suppose *A* has dimensions *p x q* , *B* has dimensions *q x r*, and *C* has dimensions *r x s* ,then

> *AB has dimensions p x r. Therefore (AB)C has dimensions p x s. The total number of operations for computing the product (AB)C is* ***p x r x s.***
>
> *Again, the product BC has dimensions q x s. Therefore, A(BC) has dimensions p x s and total number of operations in computing A(BC) is* ***p x q x s***

- Since ,in general, ***p x r x s ≠ p x q x s,*** number of operations for the product *(AB)C* is not the same as the number of operations for *A(BC)*

4

Matrix Chain Multiplication

Different Parenthesizing Orders

- Consider a chain of four matrices **A1,A2,A3,A4**, with dimensions as follows:

  A1 = p0 x p1 = 50 x 15
  A2 = p1 x p2 = 15 x 30
  A3 = p2 x p3 = 30 x 45
  A4 = p3 x p4 = 45 x 20

- The matrices can be multiplied using any of the parenthesizing orders: **((A1A2)A3)A4, A1((A2A3)A4) , (A1A2)(A3A4), A1(A2(A3A4)) .The associated number of multiplication and addition operations are listed below. The *optimal order* with *minimum cost of 48,750* is A1((A2A3)A4).**

```
CASE(1): ((A1 A2) A3) A4
=p0 x p1 x p2 +  p0 x p2 x p3 +  p0 x p3 x p4
=50 x 15 x 30 +  50 x 30 x 45 +  50 x 45 x 20
=22500 + 67500 + 45000 = 135000

CASE(2): A1((A2 A3) A4)
=p1 x p2 x p3 +  p1 x p3 x p4  +  p0 x p1 x p4
=15 x 30 x 45 +  15 x 45 x 20 +  50 x 15 x 20
= 20250 + 13500 + 15000 = 48750   <-- Lowest value

CASE(3):(A1 A2)(A3 A4)
=p0 x p1 x p2 +  p2 x p3 x p4  +  p0 x p2 x p4
=50 x 15 x 30 +  30 x 45 x 20 +  50 x 30 x 20
= 22500 + 27000 + 30000 = 79500

CASE(4):A1 (A2 (A3 A4))
=p0 x p1 x p4 +  p1 x p2 x p4  +  p2 x p3 x p4
=50 x 15 x 20 +  15 x 30 x 20 +  30 x 45 x 20
= 15000 + 9000 + 27000 = 51000

Minimum number of operations= 48,750
```

5

$A_1((A_2\ A_3\ A_4\ A_5\ A_6\ A_7\ A_8)A_9)$

n

$$\sum_{k=1}^{n-1} T(k)\,T(n-k)$$

6

Matrix Chain Multiplication

Brute Force

When the number of matrices in a chain is large, the number of ways of parenthesizing the chain increases tremendously.

- Consider the problem of splitting a chain of n matrices $A_1A_2A3.....A_n$. Suppose the chain is split into two subchains $(A_1A_2...A_k)$ and $(A_{(k+1)}.....A_n)$, where k is some integer, such that $1 \leq k < n-1$.
- If $T(n)$ is the total number of way of parenthesizing the main chain, then $T(k)$ are different ways of parenthesizing the left subchain and $T(n-k)$ are the different ways of parenthesizing the right subchain. Each split in left subchain can have corresponding $T(n-k)$ splits in the right subchain. Thus, total number of splits $T(n)$ can be obtained by considering all values of k. Thus, we have the recurrence

$$T(n) = \sum_{k=1}^{n-1} T(k).T(n-k)$$

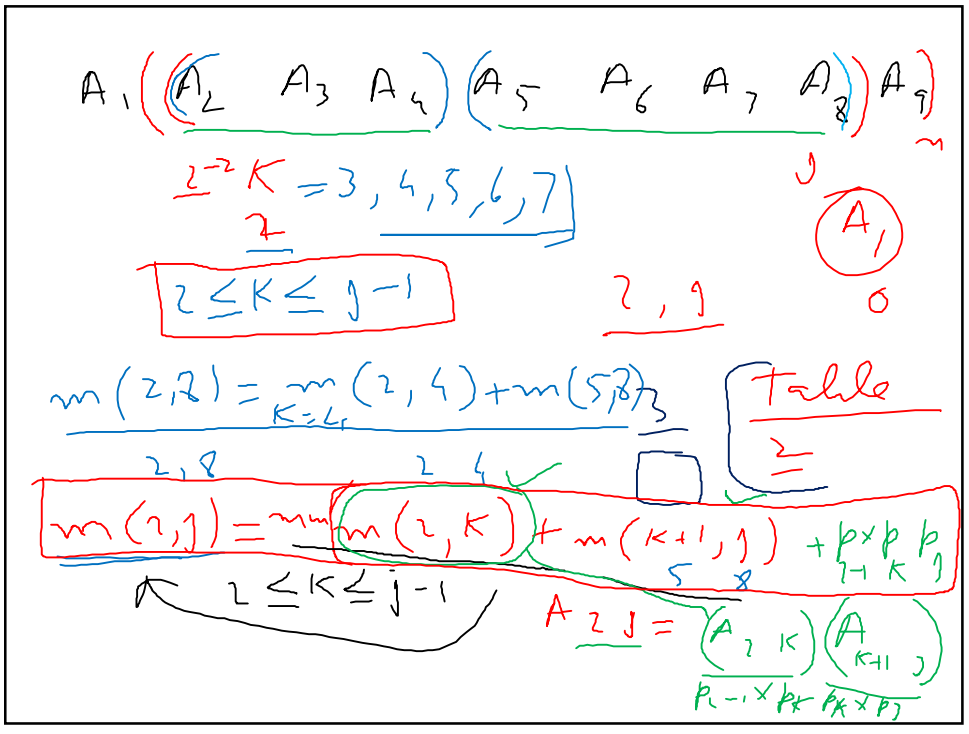
- The recurrence is similar to the recurrence for ***C(n), Catalan number,*** which gives total number of binary trees with nodes n. It can be seen that $T(n)=C(n-1)$.

- Thus, $T(n)$ is given by the formula

$$T(n) = \frac{1}{n} \frac{2(n-1)!}{(n-1)!.(n-1)!}$$

- Since growth of factorials is extremely fast, the use of brute force algorithm will not be feasible for large matrix multiplication chains.

7



8

Matrix Chain Multiplication

D P Formulation

- In order to formulate ***dynamic programming solution*** to the matrix chain multiplication, we first set up a ***recurrence for the multiplication costs***. Consider the multiplication chain consisting of matrices $A_1, A_2, A_3 \ldots\ldots A_n$, with dimensions $p_0 \times p_1$, $p_1 \times p_2$, $p_2 \times p_3 \ldots\ldots$, $p_{n-1} \times p_n$

- Let $m(i,j)$ be the ***minimum number of operations for multiplying*** the chain $A_i A_{(i+1)} \ldots A_{(j-1)} A_j$
Obviously,, $m(i,i)$=0 for all i, because the cost of multiplying a single matrix is zero.

- Suppose the chain $A_i A_{(i+1)} \ldots A_j$ is split into two subchains $A_i A_{(i+1)} \ldots\ldots A_k$ and $A_{(k+1)} \ldots A_j$ where $i \le k < j$. Since the chain $A_i A_{(i+1)} \ldots\ldots A_k$ is a matrix of dimension $p_{i-1} \times p_k$ and chain $A_{(k+1)} \ldots A_j$ is matrix of dimension $p_k \times p_j$, the total cost of multiplying the chain $A_i A_{(i+1)} \ldots A_j$ is given by

$$\underbrace{m(i,j)}_{\substack{\text{Cost of multiplying} \\ \text{the chain} \\ A_i A_{i+1} \ldots A_j}} = \underbrace{m(i,k)}_{\substack{\text{Cost of multiplying} \\ \text{first subchain} \\ A_i A_{i+1} \ldots A_k}} + \underbrace{m(k+1,j)}_{\substack{\text{Cost of multiplying} \\ \text{second subchain} \\ A_{k+1} A_{k+2} \ldots A_j}} + \underbrace{p_{i-1} p_k p_j}_{\substack{\text{Cost of multiplying} \\ \text{first and second} \\ \text{subchains}}}$$

- The ***optimal cost*** can be obtained by evaluating the cost for ***different values of k,*** and then choosing the least of the computed costs. Thus, we have recurrence for optimum cost

$$m(i, i) = 0$$
$$m(i, j) = \underset{i \le k < j}{\textbf{minimum}} \left(m(i, k) + m(k+1, j) + p_{i-1} p_k p_j \right), \quad \text{for } i < j$$

9

Matrix Chain Multiplication

D P Solution

- The following recurrence can be used to build ***dynamic programming solution in a bottom up manner***.

$$m(i, i) = 0$$
$$m(i, j) = \underset{i \le k < j}{\textbf{minimum}} \left(m(i, k) + m(k+1, j) + p_{i-1} p_k p_j \right), \quad \text{for } i < j$$

- We first consider optimal cost of multiplying a chain of ***two matrices***, then optimal cost of multiplying chains of ***three matrices,*** and ***so on***. At each step we store the costs in a table, which are used to find minimum cost at next step.

- Let w be number of matrices in the chain $A_i A_{i+1} \ldots A_j$. Then, $w=j-i+1$ or $j=w+i-1$. Since $j \le n$, $i+w-1 \le n$ or $i \le n-w+1$. Thus, in order to compute $m(i,j)$ we can set up ***two nested loops*** in which w varies from 2 to n , and i varies from $n-w+1$. The index j *equals* $w+i-1$

- Again, in order to find minimum cost ,we set up ***another loop in which k varies from i to j-1,*** which the range prescribed in the recurrence.

10

Matrix Chain Multiplication

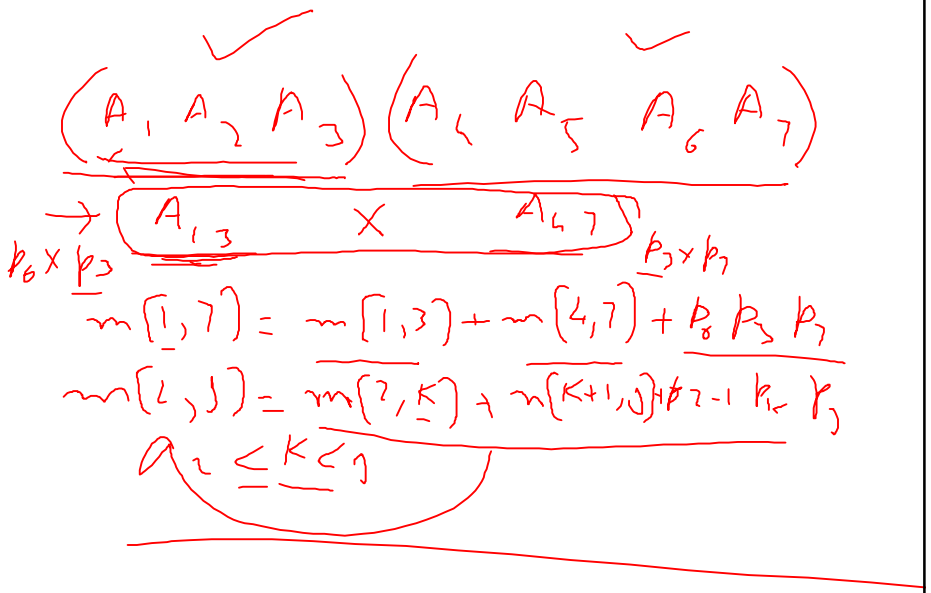
Pseudo Code

The following code determines the optimal parenthesizing for a chain of matrix multiplication. The costs are stored in array *m[i..n,j..j]* and information about optimal ordering is saved in array *s[i..n,j..n]*. The array *p[1..n]* stores the dimensions of the matrices in the chain.

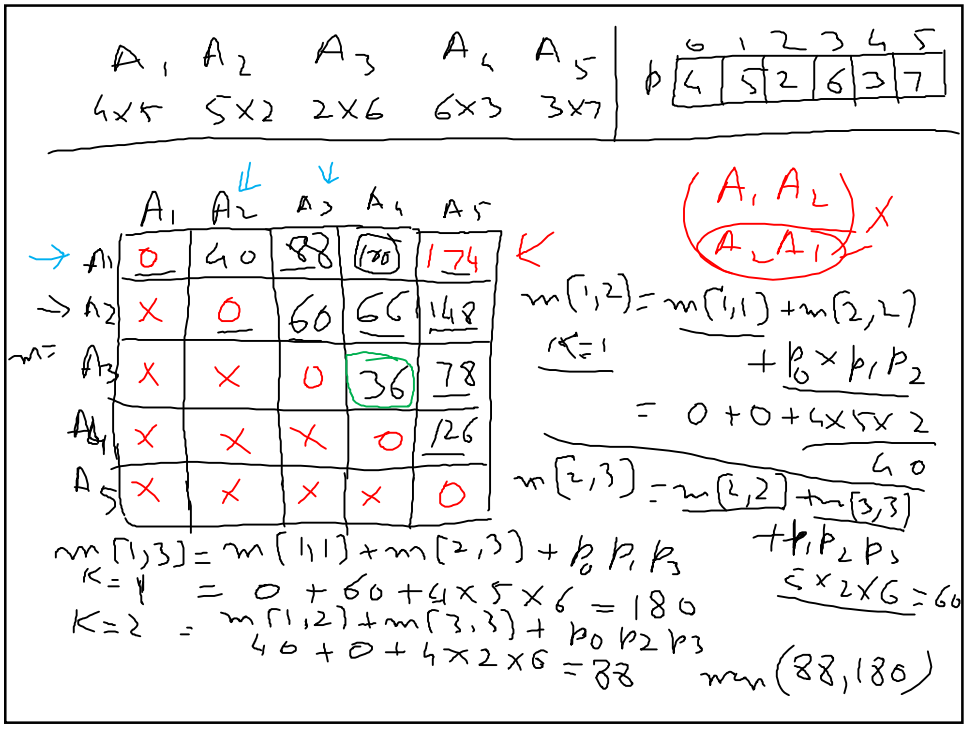
```
MATRIX-CHAIN(n, p)
for i←1 to n do
    m[i, i]←0
for w←2 to n do   ►w is the size of subchain
  for i←1 to n-w+1 do
     j←w+i -1
     m[i, j] ←∞      ► initialize the costs to a large value
    for k←i to j-1 do
      q←m[i,k] +m[k+1, j] +p[i-1]p[k]p[j]
       if q< m[i, j]
          then m[i, j]←q  ►store minimum cost
               s[i, j]←k  ►Store index of the matrix for optimum split
return m, s
```

The matrix chain multiplication has time complexity $\theta(n^3)$

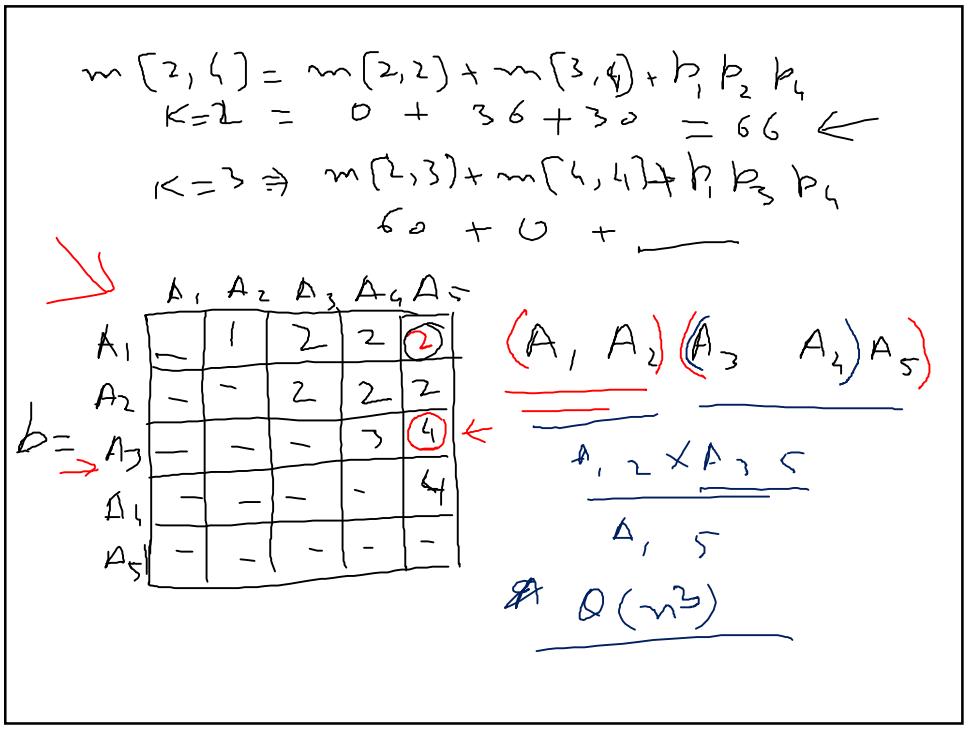
11



12


A1 A2 A3 A4 A5
4x5 5x2 2x6 6x3 3x7
p 4 5 2 6 3 7
m=
A1 0 40 88 180 174
A2 X 0 60 66 148
A3 X X 0 36 78
A4 X X X 0 126
A5 X X X X 0
m[1,3] = m[1,1] + m[2,3] + p0 p1 p3
k=1 = 0 + 60 + 4x5x6 = 180
k=2 = m[1,2] + m[3,3] + p0 p2 p3
40 + 0 + 4x2x6 = 88
m[1,2] = m[1,1] + m[2,2] + p0 p1 p2
k=1
= 0 + 0 + 4x5x2
40
m[2,3] = m[2,2] + m[3,3] + p1 p2 p3
5x2x6 = 60
(A1 A2) X
(A2 A1) X
min(88,180)

13


m[2,4] = m[2,2] + m[3,4] + p1 p2 p4
k=2 = 0 + 36 + 30 = 66
k=3 ⇒ m[2,3] + m[4,4] + p1 p3 p4
60 + 0 + ___
A1 A2 A3 A4 A5
A1 - 1 2 2 2
A2 - - 2 2 2
A3 - - - 3 4
A4 - - - - 4
A5 - - - - -
b=
(A1 A2) (A3 A4) A5
A1,2 x A3,5
A1,5
O(n^3)

14

Matrix Chain Multiplication

Listing Order

▪ The information about the ordering of parentheses is stored in the array *s[1..n,1..n]*.
For example, if *s[i,j]=k* then subchain $A_iA_{i+1}....A_j$ is ***split*** as follows

$$(A_iA_{i+1}....A_k)(A_{k+1}.....A_j)$$

Further, if *s[i,k]=q,*

$$(\ (A_iA_{i+1}..A_q)\ (A_{q+1}A_{q+2}...A_k)\ )\ \ (A_{k+1}...A_j)$$

Again if *s[k,j]=m, the chain is order as follows:*

$$(\ (A_iA_{i+1}..A_q)\ (A_{q+1}A_{q+2}...A_k)\ )\ (\ (A_{k+1}....A_m)(A_{m+1}...A_j)\ )$$

▪In this way, by using the array *s[1..n,1..n]*, the final ordering of the chain can be obtained.

15

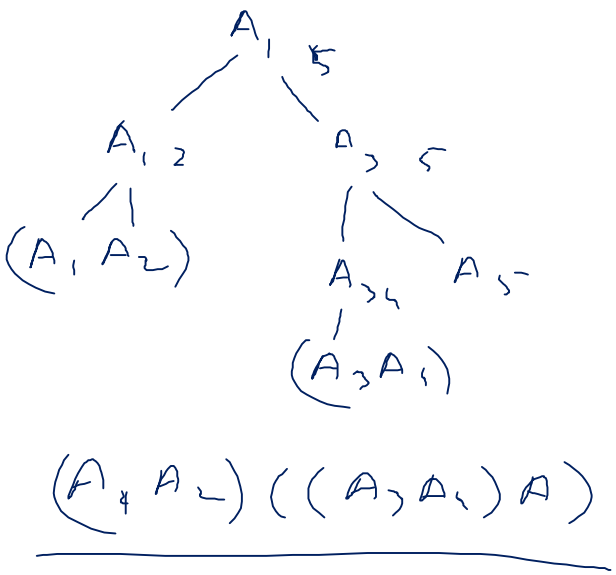
Matrix Chain Multiplication

Pseudo Code

The following code ***prints the optimal order of matrix chain multiplication***. The array *s[1..n,1..n]* stores information about the ordering. The chain is printed by recursive calls

```
ORDER(i, j, s)
If  i=j
   then print "A"i
  else  print "("
          print  ORDER(i, s[i,j], s)
          print ORDER(s[i,j]+1 , j, s)
          print ")"
```

16



17

Matrix Chain Multiplication

Comparison of DP and Brute Force Computations

▪Let $T_{brute}(n)$ be the number of computations using ***brute force algorithm*** and $T_{dp}(n)$ the number of computation using ***dynamic programming*** , to multiply a sequence of n matrices. Then,

$$T_{brute}(n) \approx \frac{2(n-1)!}{n(n-1)!\,(n-1)!}$$

$$T_{dp}(n) \approx n^3$$

▪The table shows some typical computations for the two algorithms. The running time for the brute force increases explosively.

n	$T_{brute}(n)$	$T_{dp}(n)$
10	4862	1000
20	1767263190	8000
30	1.0×10^{15}	27000
40	6.86×10^{20}	64000
50	5.0×10^{26}	125000

18